

Código C — Paradigma Imperativo

Explicação didática linha a linha — `imperativo/folha_pagamento.c`

[Folha de Pagamento](#)

[Paradigmas](#)

[Linha a linha](#)

Como ler este código

C usa comandos sequenciais, arrays, structs, laços e variáveis acumuladoras. A leitura ideal é seguir o fluxo de execução do main e das funções auxiliares.

Regras de negócio usadas

- $\text{valor_hora} = \text{salário_bruto} / 180$
- $\text{extras} = \text{valor_hora} \times 1,5 \times \text{horas extras}$
- $\text{base_irrf} = \text{bruto} + \text{extras} - \text{INSS} - \text{dependentes} \times 189,59$
- $\text{líquido} = \text{bruto} + \text{extras} - \text{INSS} - \text{IRRF}$
- Os mesmos três funcionários são usados em todas as versões: Ana, Bruno e Carla.

Visão geral da solução

A implementação produz os mesmos resultados verificados nas quatro linguagens: Total da folha = 6564.90; maior líquido = Ana (3317.07); menor líquido = Carla (1245.83).

Explicação linha a linha

Linha	Código	Explicação didática
1	<code>#include <stdio.h></code>	Importa a biblioteca padrão de entrada e saída, necessária para printf e puts.
2	<code>#include <stddef.h></code>	Importa definições como size_t, usado para índices e tamanhos de arrays.
3		Linha em branco usada para separar blocos e melhorar a leitura.
4	<code>typedef struct {</code>	Inicia a definição da estrutura Funcionario, que agrupa os dados de entrada.
5	<code>const char *nome;</code>	Guarda o nome do funcionário como texto constante.
6	<code>double salario_bruto;</code>	Guarda o salário bruto com casas decimais.
7	<code>int dependentes;</code>	Guarda a quantidade inteira de dependentes.

Linha	Código	Explicação didática
8	<code>int horas_extras;</code>	Guarda a quantidade inteira de horas extras.
9	<code>} Funcionario;</code>	Fecha a struct e cria o tipo Funcionario.
10		Linha em branco usada para separar blocos e melhorar a leitura.
11	<code>typedef struct {</code>	Linha de implementação que compõe a lógica descrita no bloco atual.
12	<code>double limite;</code>	Define o teto de uma faixa de imposto.
13	<code>double aliquota;</code>	Define a porcentagem aplicada naquela faixa.
14	<code>} Faixa;</code>	Fecha a estrutura Faixa, usada para limite e alíquota.
15		Linha em branco usada para separar blocos e melhorar a leitura.
16	<code>typedef struct {</code>	Linha de implementação que compõe a lógica descrita no bloco atual.
17	<code>const char *nome;</code>	Guarda o nome do funcionário como texto constante.
18	<code>double bruto;</code>	Campo numérico do resultado da folha para um funcionário.
19	<code>double extras;</code>	Campo numérico do resultado da folha para um funcionário.
20	<code>double inss;</code>	Campo numérico do resultado da folha para um funcionário.
21	<code>double irrf;</code>	Campo numérico do resultado da folha para um funcionário.
22	<code>double liquido;</code>	Campo numérico do resultado da folha para um funcionário.
23	<code>} Resultado;</code>	Fecha a estrutura Resultado, que reúne todos os valores calculados.
24		Linha em branco usada para separar blocos e melhorar a leitura.
25	<code>static const Faixa FAIXAS_INSS[] = {</code>	Declara a tabela fixa de faixas do INSS.

Linha	Código	Explicação didática
26	<code>{1412.00, 0.075},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
27	<code>{2666.00, 0.090},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
28	<code>{4000.00, 0.120},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
29	<code>{1.0 / 0.0, 0.140}</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
30	<code>};</code>	Fecha ou organiza o bloco iniciado anteriormente.
31		Linha em branco usada para separar blocos e melhorar a leitura.
32	<code>static const Faixa FAIXAS_IRRF[] = {</code>	Declara a tabela fixa de faixas do IRRF.
33	<code>{2259.00, 0.000},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
34	<code>{2826.00, 0.075},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
35	<code>{3751.00, 0.150},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
36	<code>{4664.00, 0.225},</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
37	<code>{1.0 / 0.0, 0.275}</code>	Entrada da tabela: limite da faixa e alíquota correspondente.
38	<code>};</code>	Fecha ou organiza o bloco iniciado anteriormente.
39		Linha em branco usada para separar blocos e melhorar a leitura.
40	<code>static double buscar_aliquota(double valor, const Faixa faixas[], size_t quantidade) {</code>	Declara função auxiliar que encontra a alíquota correta para um valor.
41	<code>for (size_t i = 0; i < quantidade; i++) {</code>	Percorre todos os elementos de um array usando índice.
42	<code>if (valor <= faixas[i].limite) {</code>	Testa se o valor cabe na faixa atual.

Linha	Código	Explicação didática
43	<code>return faixas[i].aliquota;</code>	Retorna a alíquota da primeira faixa compatível.
44	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
45	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
46	<code>return faixas[quantidade - 1].aliquota;</code>	Retorno de segurança: usa a última alíquota se nenhuma faixa anterior servir.
47	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
48		Linha em branco usada para separar blocos e melhorar a leitura.
49	<code>static double calcular_extras(const Funcionario *f) {</code>	Função que calcula o adicional por horas extras de um funcionário.
50	<code>double valor_hora = f->salario_bruto / 180.0;</code>	Guarda o salário bruto com casas decimais.
51	<code>return valor_hora * 1.5 * f->horas_extras;</code>	Calcula o valor da hora dividindo o salário bruto por 180 horas.
52	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
53		Linha em branco usada para separar blocos e melhorar a leitura.
54	<code>static Resultado calcular_funcionario(const Funcionario *f) {</code>	Função principal de cálculo individual: recebe um funcionário e devolve seu Resultado.
55	<code>double extras = calcular_extras(f);</code>	Campo numérico do resultado da folha para um funcionário.
56	<code>double aliquota_inss = buscar_aliquota(f->salario_bruto, FAIXAS_INSS, sizeof(FAIXAS_INSS) / sizeof(FAIXAS_INSS[0]));</code>	Guarda o salário bruto com casas decimais.
57	<code>double inss = f->salario_bruto * aliquota_inss;</code>	Guarda o salário bruto com casas decimais.
58	<code>double deducacao_dependentes = f->dependentes * 189.59;</code>	Calcula a dedução do IRRF multiplicando dependentes por 189,59.

Linha	Código	Explicação didática
59	<code>double base_irrf = f->salario_bruto + extras - inss - deducacao_dependentes;</code>	Guarda o salário bruto com casas decimais.
60	<code>if (base_irrf < 0.0) base_irrf = 0.0;</code>	Monta a base de cálculo do IRRF: bruto + extras - INSS - deduções.
61	<code>double aliquota_irrf = buscar_aliquota(base_irrf, FAIXAS_IRRF, sizeof(FAIXAS_IRRF) / sizeof(FAIXAS_IRRF[0]));</code>	Define a porcentagem aplicada naquela faixa.
62	<code>double irrf = base_irrf * aliquota_irrf;</code>	Campo numérico do resultado da folha para um funcionário.
63		Linha em branco usada para separar blocos e melhorar a leitura.
64	<code>Resultado r = {f->nome, f->salario_bruto, extras, inss, irrf, f->salario_bruto + extras - inss - irrf};</code>	Monta a struct Resultado com todos os valores calculados, incluindo líquido.
65	<code>return r;</code>	Devolve o resultado individual calculado.
66	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
67		Linha em branco usada para separar blocos e melhorar a leitura.
68	<code>static void imprimir_resultado(Resultado r) {</code>	Monta a struct Resultado com todos os valores calculados, incluindo líquido.
69	<code>printf("%-8s Bruto: %8.2f Extras: %7.2f INSS: %7.2f IRRF: %7.2f Líquido: %8.2f\n",</code>	Imprime valores monetários com duas casas decimais e alinhamento.
70	<code> r.nome, r.bruto, r.extras, r.inss, r.irrf, r.líquido);</code>	Linha de implementação que compõe a lógica descrita no bloco atual.
71	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
72		Linha em branco usada para separar blocos e melhorar a leitura.
73	<code>int main(void) {</code>	Ponto de entrada do programa em C.
74	<code>Funcionario funcionarios[] = {</code>	Cria o array com os três funcionários do caso de teste.
75	<code> {"Ana", 4500.00, 2, 8},</code>	Um funcionário de teste: nome, bruto, dependentes e horas extras.

Linha	Código	Explicação didática
76	<code>{"Bruno", 2200.00, 0, 0},</code>	Um funcionário de teste: nome, bruto, dependentes e horas extras.
77	<code>{"Carla", 1300.00, 1, 4}</code>	Um funcionário de teste: nome, bruto, dependentes e horas extras.
78	<code>};</code>	Fecha ou organiza o bloco iniciado anteriormente.
79	<code>size_t quantidade = sizeof(funcionarios) / sizeof(funcionarios[0]);</code>	Calcula automaticamente quantos funcionários existem no array.
80		Linha em branco usada para separar blocos e melhorar a leitura.
81	<code>double total = 0.0;</code>	Inicializa o acumulador do total da folha.
82	<code>Resultado maior = calcular_funcionario(&funcionarios[0]);</code>	Inicializa o maior salário líquido com o primeiro funcionário.
83	<code>Resultado menor = maior;</code>	Inicializa o menor salário líquido com o mesmo primeiro resultado.
84		Linha em branco usada para separar blocos e melhorar a leitura.
85	<code>puts("Folha de pagamento - paradigma imperativo (C)");</code>	Exibe texto simples de cabeçalho ou política de cálculo.
86	<code>puts("Politica: valores monetarios exibidos com 2 casas decimais; calculos internos usam double.\n");</code>	Exibe texto simples de cabeçalho ou política de cálculo.
87		Linha em branco usada para separar blocos e melhorar a leitura.
88	<code>for (size_t i = 0; i < quantidade; i++) {</code>	Percorre todos os elementos de um array usando índice.
89	<code>Resultado atual = calcular_funcionario(&funcionarios[i]);</code>	Calcula o resultado do funcionário atual do laço.
90	<code>imprimir_resultado(atual);</code>	Função de apresentação: imprime um resultado formatado.
91	<code>total += atual.liquido;</code>	Soma o líquido atual ao total da folha.
92	<code>if (atual.liquido > maior.liquido) maior = atual;</code>	Atualiza o maior líquido se o funcionário atual superar o maior salvo.

Linha	Código	Explicação didática
93	<code>if (atual.liquido < menor.liquido) menor = atual;</code>	Atualiza o menor líquido se o funcionário atual for menor que o salvo.
94	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.
95		Linha em branco usada para separar blocos e melhorar a leitura.
96	<code>printf("\nTotal da folha: %.2f\n", total);</code>	Imprime valores monetários com duas casas decimais e alinhamento.
97	<code>printf("Maior salario liquido: %s (%.2f)\n", maior.nome, maior.liquido);</code>	Imprime valores monetários com duas casas decimais e alinhamento.
98	<code>printf("Menor salario liquido: %s (%.2f)\n", menor.nome, menor.liquido);</code>	Imprime valores monetários com duas casas decimais e alinhamento.
99	<code>return 0;</code>	Encerra o programa informando sucesso ao sistema operacional.
100	<code>}</code>	Fecha ou organiza o bloco iniciado anteriormente.

Resumo para apresentação oral

Explique que C deixa o fluxo claro: define estruturas, percorre arrays com for, usa acumuladores para total e comparações para maior/menor. O ponto de atenção é gerenciar manualmente índices, arrays e mutação.